SLAM-LLM: A Modular, Open-Source Multimodal Large Language Model Framework and Best Practice for Speech, Language, Audio and Music Processing

Ziyang Ma, Guanrou Yang, Wenxi Chen, Zhifu Gao, Yexing Du, Xiquan Li, Zhisheng Zheng, Haina Zhu, Jianheng Zhuo, Zheshu Song, Ruiyang Xu, Tiranrui Wang, Yifan Yang, Yanqiao Zhu, Zhikang Niu, Liumeng Xue, Yinghao Ma, Ruibin Yuan, Shiliang Zhang, Kai Yu, Eng Siong Chng, Xie Chen

***Abstract*—The recent surge in open-source Multimodal Large Language Models (MLLM) frameworks, such as LLaVA, provides a convenient kickoff for artificial intelligence developers and researchers. However, most of the MLLM frameworks take vision as the main input modality, and provide limited in-depth support for the modality of speech, audio, and music. This situation hinders the development of audio-language models, and forces researchers to spend a lot of effort on code writing and hyperparameter tuning. We present SLAM-LLM, an open-source deep learning framework designed to train customized MLLMs, focused on speech, language, audio, and music processing. SLAM-LLM provides a modular configuration of different encoders, projectors, LLMs, and parameter-efficient fine-tuning plugins. SLAM-LLM also includes detailed training and inference recipes for mainstream tasks, along with high-performance checkpoints like LLM-based Automatic Speech Recognition (ASR), Automated Audio Captioning (AAC), and Music Captioning (MC). Some of these recipes have already reached or are nearing state-of-the-art performance, and some relevant techniques have also been accepted by academic papers. We hope SLAM-LLM will accelerate iteration, development, data engineering, and model training for researchers. We are committed to continually pushing forward audio-based MLLMs through this open-source framework, and call on the community to contribute to the LLM-based speech, audio and music processing.**

***Index Terms*—Multimodal Large Language Model, Large Language Model, Framework, Toolkit, Speech Processing, Language Processing, Audio Processing, Music Processing**

I. Introduction

The rapid advancement of Large Language Models (LLMs) has catalyzed the development of multimodal learning systems that integrate various forms of input such as text, vision, as well as audio. While recent open-source frameworks, such as LLaVA [1] and OpenFlamingo [2] have demonstrated remarkable capabilities in vision-language modeling, they remain largely vision-centric. These frameworks offer limited support for non-visual modalities, particularly in the areas of speech, audio, and music processing. This lack of native support presents significant challenges for researchers working on auditory tasks, often forcing them to retrofit vision-based systems through cumbersome adaptations, resulting in inefficiencies and fragmented development workflows.

To address these limitations, we introduce **SLAM-LLM: a modular, open-source framework for Multimodal Large Language Models (MLLMs), with a specific focus on speech, audio, and music.** SLAM-LLM is designed to lower the barrier for constructing, training, and deploying LLM-based systems that process audio-related modalities. Its core architecture follows a clean encoder–projector–LLM modular design, enabling seamless customization of model components through YAML configuration files. The framework supports a wide range of pretrained encoders (e.g., Whisper [3], HuBERT [4], BEATs [5], MERT [6]), projection modules (MLP layers, CNN layers, Q-Former [7]), and LLM backbones (e.g., LLaMA [8], [9], Vicuna [10], Qwen [11]). It also incorporates parameter-efficient fine-tuning (PEFT) strategies such as Low-Rank Adaptation (LoRA) and prefix-tuning, making it suitable for low-resource and domain-specific adaptation. SLAM-LLM's design also addresses common engineering challenges in multimodal modeling, such as handling variable-length inputs and efficient fine-tuning, while supporting rapid prototyping and reproducibility.

Beyond architectural flexibility, SLAM-LLM provides a comprehensive suite of training recipes and pretrained checkpoints for a variety of key tasks: Automatic Speech Recognition (ASR), Speech-to-Text Translation (S2TT), Speech Emotion Captioning (SEC), Automated Audio Captioning (AAC), and Music Captioning (MC). Extensive experiments across these tasks reveal a series of key insights.

In brief, SLAM-LLM makes several significant contributions:

1) A unified, flexible, and extensible framework for developing LLM-based models for audio-related modalities;
2) Extensive modular support for diverse encoders, projectors, and large language models;

Ziyang Ma, Guanrou Yang, Wenxi Chen, Xiquan Li, Haina Zhu, Jianheng Zhuo, Zheshu Song, Ruiyang Xu, Yifan Yang, Yanqiao Zhu, Zhikang Niu, Kai Yu, and Xie Chen are with the X-LANCE Lab, School of Computer Science, MoE Key Lab of Artificial Intelligence Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: {zym.22, yangguanrou, 1029713857, mtxiaoxi55, hainazhu, zzasdf, songzheshu, xry2022, yifanyeung, 1850432206, zhikangniu, kai.yu, chenxie95}@sjtu.edu.cn). Zhifu Gao is with the Tongyi Lab, Alibaba Group, Hangzhou 311121, China. (e-mail: zhifu.gzf@alibaba-inc.com). Yexing Du is with the Peng Cheng Laboratory, Guangdong 518055, China. (e-mail: yxdu@ir.hit.edu.cn). Zhisheng Zheng is with the University of Texas at Austin, Texas 78712, United States. (e-mail: zszheng@utexas.edu). Tianrui Wang is with the Tianjin University, Tianjin 300350, China. (wangtianrui@tju.edu.cn). Liumeng Xue and Ruibin Yuan are with the Hong Kong University of Science and Technology, Hongkong 999077, China. (e-mail: {lmxue, ryuanab}@ust.hk). Yinghao Ma is with the Queen Mary University of London, East London E1 4NS, England. (e-mail: yinghao.ma@qmul.ac.uk). Shiliang Zhang is with the University of Science and Technology of China, Hefei 230052, China. (e-mail: zsl2008@ustc.edu.cn). Eng Siong Chng is with the Nanyang Technological University, 639956, Singapore. (e-mail: ASESChng@ntu.edu.sg).

Corresponding author: Prof. Xie Chen

Open source at https://github.com/X-LANCE/SLAM-LLM

3) A library of curated training and inference recipes across speech, audio, and music tasks;
4) State-of-the-art results across benchmarks, particularly in speech recognition and audio captioning;
5) Open-source implementation encouraging community collaboration and rapid innovation.

SLAM-LLM bridges a major gap in the current MLLM ecosystem and promotes the development for future progress in general-purpose audio-language models (ALMs). By open-sourcing the framework and accompanying recipes, SLAM-LLM invites the broader community to contribute to and accelerate innovation in this important yet limitedly developed area of multimodal AI.

II. Multimodal Large Language Model Framework

The development of LLMs has catalyzed a rapidly expanding ecosystem of open-source toolkits designed to support the training, fine-tuning, and deployment of LLMs across a variety of modalities. In particular, the recent surge of interest in multimodal learning has motivated the emergence of several frameworks that extend LLMs beyond pure text understanding.

Among these toolkits, a number of general-purpose LLM infrastructure frameworks have been proposed to facilitate model development and customization. Notably, LLaMA-recipes [12], an official collection of implementations and usage patterns for the LLaMA model family released by Meta, provides end-to-end support for both text-based and visual-language models. LLaMA-Adapter [13] introduces a parameter-efficient tuning strategy for adapting frozen LLaMA models to instruction-following and, in its extended version, visual modalities, by introducing a vision-language alignment mechanism. LLaMA2-Accessory [14] further generalizes this line of work, offering a comprehensive toolbox for pretraining, fine-tuning, and inference across both unimodal and multimodal settings, with native support for integrating visual encoders such as CLIP [15] and DINOv2 [16]. In parallel, LLaMA-Factory [17] provides a unified and extensible training interface for over 100 foundation models, including multimodal branches that support image, audio, and video tasks via task-specific adapters and fine-tuning templates. Additionally, LMFlow [18] and LitGPT offer efficient solutions for finetuning and inference, focusing on memory optimization, scalability, and support for multimodal inputs.

In contrast to these infrastructure-focused toolkits, another line of work has produced end-to-end multimodal LLMs designed specifically for vision-language models (VLMs). MiniGPT-4 [19] demonstrates that aligning a pretrained visual encoder with a frozen language model using a lightweight projection layer enables powerful vision-language dialogue capabilities. LLaVA [1] adopts a two-stage training strategy that combines visual feature alignment with instruction tuning, resulting in a highly capable open-source visual dialogue model. TinyLLaVA [20] builds on this approach with a focus on model efficiency, achieving competitive multimodal performance with significantly reduced parameter counts. Meanwhile, OpenFlamingo [2] reimplements DeepMind's Flamingo [21] architecture for open access, supporting few-shot learning across sequences of interleaved image-text pairs and offering pre-trained models at various scales.

However, most existing toolkits primarily target text or vision tasks, offering limited or no support for non-visual modalities such as speech, audio, and music. As a result, researchers in auditory domains—e.g., speech recognition, audio captioning, or music analysis—must often repurpose tools whose architectures and pipelines are tightly coupled to vision or text, requiring substantial adaptation effort. This lack of native support hinders progress in building efficient audio-language models. While toolkits like ESPnet[1] focus on end-to-end speech systems and Fairseq[2] focus on speech sequence modeling, they do not adopt an LLM-centric design, which is a paradigm shift in SLAM-LLM. The framework addresses this gap with a modular design that seamlessly integrates speech, audio, and music encoders with LLMs. In SLAM-LLM, all auditory tasks are unified into an auto-regressive generation process, allowing researchers to leverage the strong text power of LLMs while modularly incorporating proven speech processing best practices to guide the generative performance. Its flexible architecture supports easy customization of encoders, projectors, and LLMs, and is compatible with PEFT [22], FSDP [23], and DeepSpeed [24]. With comprehensive training and inference recipes, SLAM-LLM accelerates research and promotes community-driven development via its open-source release.

III. Design of SLAM-LLM

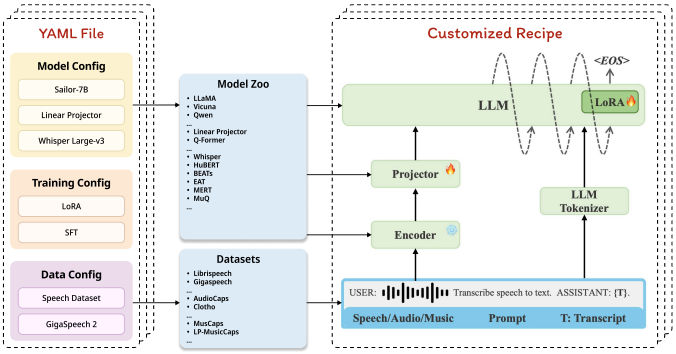

Fig. 1: A brief workflow in the SLAM-LLM framework. By specifying model, training, and data configurations in a YAML file, customized requirements are met efficiently. The depicted example demonstrates an LLM-based Southeast Asian ASR model using a Southeast Asian language LLM Sailor-7B and corresponding speech dataset GigaSpeech 2.

A. Overview of SLAM-LLM

SLAM-LLM allows users to configure customized MLLMs through a YAML file, significantly reducing the burden on model construction. Users specify the necessary model components, training strategies, and data formats in the YAML file, enabling SLAM-LLM to tailor the training and inference according to specific customized requirements.

[1] https://github.com/espnet/espnet
[2] https://github.com/facebookresearch/fairseq

Figure 1 exemplifies how SLAM-LLM operates using an LLM-based low-resource language ASR model. Specifically, during training, the YAML file designates Whisper Large-v3 [3] as the speech encoder, a linear layer as the projector, and the Southeast Asian language model Sailor-7B [25] as the LLM, using the GigaSpeech 2 [26] dataset for LoRA fine-tuning. This setup effectively configures an LLM-based Southeast Asian ASR model. For the trained model, only the parameters tuned (such as the projector and LoRA) need to be saved, and treated as generated assets. During inference, other components can be fetched from the Model Zoo and assembled along with generated assets according to the configuration, greatly enhancing flexibility in the process of development.

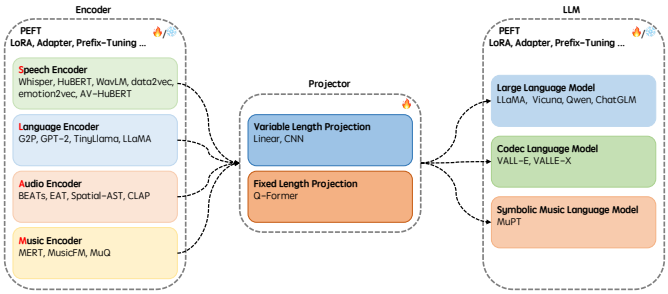

Fig. 2: SLAM-LLM modularizes the Encoder-Projector-LLM three-part components, which are assembled according to the configuration at training and inference time.

B. Modular Design

A distinguishing feature of SLAM-LLM is its modular design, which qualifies it as a framework rather than a mere model. As illustrated in Figure 2, SLAM-LLM decomposes the MLLM into three decoupled components: Encoder, Projector, and LLM. Each component can be individually specified if and how to be tuned according to the training configuration.

More specifically, the encoder serves as a perceiver of signals, supporting various types of encoders for speech, language, audio, and music. The projector bridges the encoder and the LLM. SLAM-LLM provides support for variable-length projection, such as the linear projector, as well as fixed-length projection like Q-Former. The LLM leverages its extensive knowledge and generation capabilities to produce high-quality outputs. Within SLAM-LLM, diverse types of language models are supported, including text language models, codec language models, and symbolic music language models, catering to the customized needs in the fields of speech, audio, and music.

Based on the modular design of SLAM-LLM, we have provided various recipes and checkpoints for implementing different tasks. In the following experiment sections, we present fundamental results for most classic tasks in detail, prioritize using mainstream model components wherever possible, and conduct large-scale experiments. By comparing and analyzing various experimental results, we derive several empirical insights. These insights not only guide the optimization of existing systems but also lay the foundation for developing new components. Furthermore, these results can serve as a robust baseline reference for future research and applications.

C. Task Support

While SLAM-LLM is primarily designed to facilitate LLM-based understanding tasks (i.e., mapping audio inputs to natural language text), it has been extended to support speech generation and multi-turn dialogue to further its utility as a general-purpose framework. For generation tasks, the framework supports codec-based text-to-speech (TTS) models such as VALL-E X [27], leveraging its native support for codec language models. Furthermore, SLAM-LLM incorporated SLAM-Omni [28], an end-to-end, multi-turn speech dialogue system, processing continuous speech representations and generating discrete speech tokens as output. Such a design demonstrates the framework's flexibility in supporting complex, interaction-oriented applications that go beyond static tasks like automated audio captioning or speech recognition.

IV. LLM-based Speech Processing

A. Automatic Speech Recognition (ASR)

a) Model setup: We investigate two types of speech encoders: supervised models trained on large-scale speech-text pairs and self-supervised models trained on unlabeled speech. For supervised encoders, we use the Whisper [3] family (*tiny* to *large-v2*), retaining only the encoder as a feature extractor. For self-supervised encoders, we evaluate HuBERT [4] and WavLM [29] at multiple scales, including both pre-trained and fine-tuned versions. Base models are trained on the 960-hour LibriSpeech [30], while larger ones use LibriLight [31] (60k hours for HuBERT; 94k hours + VoxPopuli [32] + GigaSpeech [33] for WavLM). HuBERT also includes X-Large variants, among the largest publicly available. For LLMs, we consider both pre-trained and chat-based variants with supervised fine-tuning and RLHF. The pre-trained models include *TinyLLaMA* (1.1B) [34] and *LLaMA-2* (7B) [9]; chat models include *TinyLLaMA-Chat* (1.1B), *LLaMA-2-Chat* (7B), and *Vicuna* (7B) [10]. All encoder outputs are at 50 Hz and downsampled to 10 Hz before feeding into the LLM. The projector uses a hidden size of 2048; encoder output and LLM input dimensions vary by model. The training recipes and model checkpoints can be found here[3]. To address the practical requirements for real-world deployment, we report the compute resources for a representative task using a 7B-scale LLM backbone: while training configurations may vary slightly between experiments to ensure peak performance, the underlying resource requirements remain similar. Training an LLM-based model on 1,000 hours of speech data for 1 epoch, while keeping the LLM and encoder frozen and updating only the projector, requires approximately 4 hours using 4 × NVIDIA A100 (80GB) GPUs.

b) Datasets: We use the LibriSpeech [30] benchmark with 960 hours of training data, without augmentation or splicing. Validation and testing are conducted on the 10-hour dev-other, test-clean, and test-other subsets.

[3] https://github.com/X-LANCE/SLAM-LLM/tree/main/examples/asr_librispeech

TABLE I: Different combinations of supervised speech encoders and LLMs to conduct LLM-based ASR. We show results of Whisper models with different sizes on LLMs of different scales.

Speech Encoder	Pre-trained Model				Chat Model					
	TinyLLaMA-1.1B		LLaMA-2-7B		TinyLLaMA-Chat-1.1B		LLaMA-2-Chat-7B		Vicuna-v1.5-7B	
	clean	other	clean	other	clean	other	clean	other	clean	other
Whisper-tiny	12.72	21.64	16.16	25.17	9.55	21.01	8.97	18.77	7.07	16.01
Whisper-base	7.35	15.89	17.46	21.84	7.03	15.92	6.37	12.98	5.07	13.07
Whisper-small	6.61	11.81	6.41	10.88	5.94	11.5	4.51	8.94	4.19	9.50
Whisper-medium	4.65	8.95	3.35	6.10	5.01	8.67	2.71	6.37	2.72	6.79
Whisper-large	4.39	8.22	3.01	7.15	4.33	8.62	2.72	6.79	2.58	6.47

c) Experiments on different supervised speech encoders: Table I presents ASR results across various combinations of supervised speech encoders and LLMs, highlighting several key findings. First, ASR performance improves with larger Whisper encoders. For example, using TinyLLaMA-1.1B, the test-clean WER drops from 12.72 (Whisper-tiny) to 4.39 (Whisper-large), showing that larger encoders better capture speech features. However, returns diminish with size: the WER improvement from Whisper-medium (4.65) to Whisper-large (4.39) is marginal. Second, chat models consistently outperform pre-trained LLMs across all encoder sizes. For instance, with Whisper-large, LLaMA-2-7B yields a test-clean WER of 3.01, while LLaMA-2-Chat-7B improves to 2.72. This suggests SFT enhances the model's ability to treat speech embeddings as a form of "language", benefiting from translation-like learning during fine-tuning. Finally, with Whisper-large fixed, Vicuna-7B achieves the best results, reaching a 2.58 WER on test-clean—surpassing both LLaMA-2-7B (3.01) and LLaMA-2-Chat-7B (2.72). This indicates that Vicuna, fine-tuned on user-shared conversational data, generalizes more effectively.

d) Experiments on different self-supervised speech encoders: Table II analyzes self-supervised speech encoders for LLM-based ASR, using Vicuna-7B-v1.5 as the fixed LLM. WER generally improves with larger encoder sizes, mirroring trends observed with supervised models. At the base scale (95M parameters, 768 hidden size), HuBERT and WavLM perform on par with Whisper-small, showing no clear advantage for self-supervised learning. However, at 300M parameters, WavLM Large surpasses all supervised encoders in TableI, including Whisper-medium (300M) and Whisper-large (600M), while HuBERT's gains from Base to Large are more modest. Fine-tuning HuBERT Large on LibriSpeech yields substantial improvements, achieving 2.10% WER on test-clean and 4.26% on test-other—outperforming WavLM Large. Scaling further to HuBERT X-Large (1B parameters, fine-tuned) delivers the best results: 1.84% WER on test-clean and 3.39% on test-other, representing relative reductions of 28.7% and 47.4% over Whisper-large. These findings underscore the strong benefits of scaling and fine-tuning SSL encoders for LLM-based ASR.

e) Comparison with Non-LLM-based ASR models: Table III compares our LLM-based ASR model (SLAM-LLM) with state-of-the-art NN-based systems, including specialist models trained on LibriSpeech-960, self-supervised models pre-trained on large-scale unlabeled speech, and universal models trained on multilingual labeled datasets. For LibriSpeech-960 specialist models, we compare against ContextNet [35], Conformer [36], Branchformer [37] (ESPnet), and Zipformer [38] (K2). These models use extensive system-level techniques—SpecAugment, speed perturbation, EMA averaging—and often fuse in-domain LMs trained on LibriSpeech text. Despite forgoing such engineering, our LLM-based model outperforms the best of these specialist systems. However, SSL models like HuBERT-large/x-large implemented with Fairseq and WavLM-large implemented with UniSpeech, pre-trained on larger unlabeled datasets and paired with in-domain LMs, outperform our model on the noisy test-other subset. These gains are largely attributable to LM integration, suggesting that domain-specific LLM fine-tuning may offer similar improvements, albeit at the cost of generalization. Compared to general-purpose models, our system surpasses Whisper-large-v2, the stronger Whisper-large-v3, and OWSM-v3.1, a top academic baseline. These results highlight the effectiveness of our LLM-based approach, achieving competitive or superior ASR performance without heavy engineering or massive pre-training.

TABLE II: Explore the performance with different SSL speech encoders for LLM-based ASR. LS-960 means the Librispeech 960 hours dataset. FT donates Fine-tuning.

Speech Encoder	Encoder Params	Hidden Size	Projector Params	WER(%) ↓ test-clean	test-other
HuBERT Base	94.70M	768	16.26M	4.43	10.72
WavLM Base	94.38M	768	16.26M	4.14	9.66
HuBERT Large	316.61M	1024	18.88M	4.53	8.74
+ LS-960 FT	316.61M	1024	18.88M	2.10	4.26
WavLM Large	315.45M	1024	18.88M	2.13	4.73
+ LS-960 FT	315.45M	1024	18.88M	1.96	4.18
HuBERT X-Large	964.32M	1280	21.50M	2.41	4.49
+ LS-960 FT	964.32M	1280	21.50M	1.84	3.39

f) Experiments on minority language datasets: Beyond high-resource languages like English and Chinese, low-resource speech recognition remains a major challenge. This experiment targets minority Southeast Asian languages: Thai (th), Vietnamese (vi), and Indonesian (id), using GigaSpeech 2 [26] for both training and evaluation. To mimic real-world low-resource conditions, training data is limited to 200 hours per language. Our model uses the Whisper large-v3 encoder [3], ReLU-activated MLP layers as the projector, and Sailor-7B [25] as the LLM. To improve performance, we partially fine-tune the encoder by freezing its first 30 layers and updating only the final two. For the LLM, we apply LoRA adapters to the query and value projections in each self-attention layer. Table IV reports results on low-resource ASR tasks. For Thai and Vietnamese, the LLM-based model (Whisper encoder + Sailor-7B) outperforms both Whisper and Whisper-finetune. In Indonesian, while it falls short of Whisper-finetune, it still

TABLE III: Compared to prior NN-based models on LibriSpeech, ***Specialist Models*** are trained solely on the 960-hour LibriSpeech dataset, often using ***in-domain LMs*** built from LibriSpeech text and transcripts. ***SSL-based Models*** are self-supervised models pre-trained on large-scale unlabeled data, then fine-tuned with supervision. ***Universal Models*** refer to general-purpose systems trained on massive labeled speech-text pairs. Repository links are provided for reproducibility. "Ours" denotes results from our best system in Table II.

Model	Implementation	Speech Data(h) Unlabeled/Labeled	WER(%) ↓ test-clean	test-other
Specialist Models				
ContextNet-large [35]	-	-/960	2.1	4.6
+ in-domain LM			1.9	4.1
Conformer-large [36]	-	-/960	2.1	4.3
+ in-domain LM			1.9	3.9
Branchformer-large [37]	ESPnet	-/960	2.4	5.5
+ in-domain LM			2.1	4.5
Zipformer-large [38]	K2	-/960	2.0	4.4
+ in-domain LM			1.9	3.9
SSL-based Models				
wav2vec 2.0-large [39]	Fairseq	60k/960	2.2	4.5
+ in-domain LM			1.8	3.3
HuBERT-large [4]	Fairseq	60k/960	2.2	4.5
+ in-domain LM			1.9	3.3
HuBERT-x-large [4]	Fairseq	60k/960	2.0	3.7
+ in-domain LM			1.8	2.9
WavLM-large [29]	UniSpeech	94k/960	2.7	5.0
+ in-domain LM			1.8	3.2
Universal Models				
Whisper-large-v2 [3]	OpenAI/Whisper	-/680k	2.7	5.2
Whisper-large-v3 [3]	OpenAI/Whisper	-/1000k	2.0	3.9
OWSM-v3.1 [40]	ESPnet	-/180k	2.4	5.0
LLM-based Models				
Ours	SLAM-LLM	-/960	1.8	3.4

surpasses the base Whisper model. These results suggest that incorporating LLMs effectively leverages rich textual knowledge to improve ASR under limited-resource settings. Sailor-7B, fine-tuned for Southeast Asian languages, further enhances performance. Compared to Vicuna-based models, the Sailor-based system achieves better results across all three languages, underscoring the value of language-specific LLM selection in low-resource scenarios.

TABLE IV: Comparison of Whisper and LLM-based ASR in WER/CER for low resource languages among Thai(th), Vietnamese(vi), and Indonesian(id). "+ Fine-tuning" refers to the model that is fine-tuned using the same amount low-resource language training data compared to the LLM-based ASR models.

Model	Language (WER%) ↓ th	vi	id
Whisper Enc. + Whisper Dec.	20.44	17.94	20.03
+ Fine-tuning	17.08	16.15	**17.13**
Whisper Enc. + Vicuna-7B	17.25	17.95	19.83
Whisper Enc. + Sailor-7B	**16.13**	**15.10**	19.48

g) Experiments on the code switching dataset: We also conducted experiments on the ASRU 2019 Mandarin-English Code-Switching Challenge dataset [41], which includes 200 hours of code-switching speech and 500 hours of Mandarin-only speech. In this study, we use only the 200-hour code-switching subset for training and a 20-hour test set for evaluation. Our system uses Whisper large-v3 as the speech encoder, Qwen2-7B as the LLM decoder, and a lightweight linear projector to align speech and text modalities. To enhance performance, LoRA adapters are applied to the query and value matrices in each self-attention layer of the LLM. Table V shows that the LLM-based model outperforms Whisper large-v3 across all metrics on the code-switching test set, achieving a 20.2% relative reduction in MER, highlighting the effectiveness of the LLM-based architecture.

TABLE V: Comparison of Whisper large-v3 and LLM-based ASR in Mandarin-English code switching ASR. MER denotes mixed error rate for both Chinese character and English words.

Model	CN CER	EN WER	MER
Whisper large-v3	7.85	35.18	10.01
Ours	**5.97**	**26.84**	**7.99**

B. Contextual Automatic Speech Recognition (CASR)

1) Task setup: We investigate two types of Contextual ASR tasks: **Visual Contextual Speech Recognition** and **Contextual Biasing Speech Recognition**, as shown in Figure 3. In Visual Contextual ASR, textual keywords extracted from presentation slides are used to improve transcription accuracy for conference content. Each speech segment is paired with pre-processed OCR results and slide-specific keywords. In Contextual Biasing ASR, a predefined list of hundreds to thousands of biasing terms—such as named entities, contact names, or song titles—is provided to help recognize rare or domain-specific vocabulary.

2) Visual Contextual Speech Recognition:

a) Datasets: We use the SlideSpeech [42] dataset for training and evaluation. This large-scale audio-visual corpus is constructed from YouTube conference videos and provides high-quality transcribed speech aligned with synchronized slides. It includes 720p videos, 16kHz audio, pre-processed OCR results, and extracted keywords per segment, supporting multimodal ASR tasks. Two training sets are available: a large set (L95) with 473 hours and a smaller subset (S95) with 161 hours. The development and test sets contain 5.07 and 8.75 hours, respectively. The dataset's alignment between speech and slides makes it well-suited for text-enhanced multimodal ASR, particularly in correcting domain-specific or proprietary terms often misrecognized by conventional systems.

b) Model setup: We adopt the official WavLM Large model as the speech encoder, Vicuna-7B as the LLM decoder, and a lightweight linear projector to align speech features with the LLM input space. The projector first downsamples 50Hz features to 10Hz via a 1D convolution, followed by two linear layers with a hidden dimension of 2048. The training recipes and model checkpoints can be found here[4].

c) Experiments: Table VI reports performance on the SlideSpeech dataset using WER, biased WER (B-WER), unbiased WER (U-WER), and keyword Recall. Our baseline model, trained on L95/S95, achieves WERs of 9.4%/11.7%, showing relative reductions of 27.9%/44.7% over the SlideSpeech

[4]https://github.com/X-LANCE/SLAM-LLM/tree/main/examples/mala_asr_slidespeech

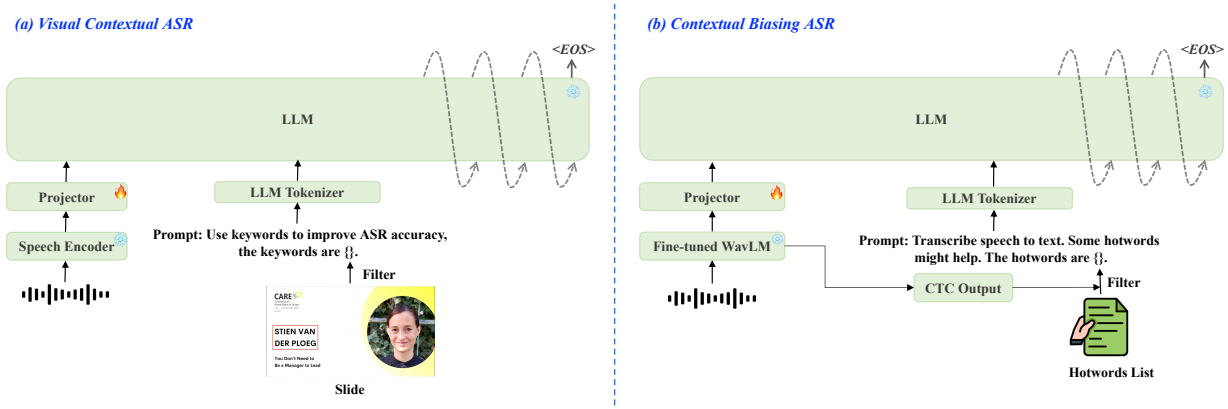

Fig. 3: (a) LLM-based Visual Contextual ASR in SLAM-LLM framework. (b) LLM-based Contextual Biasing ASR in SLAM-LLM framework.

TABLE VI: ASR results of with or without contextual keywords evaluated on dev/test datasets, trained on S95/L95 datasets, compared with previous NN-based models.

Train	Model	Contextual Keywords	Dev				Test			
			WER	B-WER	U-WER	Recall ↑	WER	B-WER	U-WER	Recall ↑
S95 (161h)	SlidesSpeech [42]	✗	21.05	31.27	20.29	68.76	21.22	26.60	20.83	73.51
	CPP [42]	✓	20.80	28.61	20.22	71.48	20.95	24.05	20.73	76.10
	LCB-net [43]	✓	18.80	27.90	18.11	72.09	19.21	23.70	18.89	76.48
	Ours	✗	11.57	16.23	**11.28**	83.83	11.80	13.52	11.71	86.71
	Ours	✓	**11.14**	**8.92**	11.36	**91.44**	**11.26**	**7.67**	**11.52**	**92.50**
L95 (473h)	SlidesSpeech [42]	✗	13.09	16.13	12.87	83.90	12.89	12.70	12.90	87.43
	CPP [42]	✓	12.64	12.39	12.66	87.64	12.38	9.32	12.60	90.86
	LCB-net [43]	✓	12.21	12.12	12.21	87.98	12.02	9.03	12.24	91.12
	Ours	✗	9.38	11.98	9.19	88.08	9.34	9.52	9.33	90.64
	+ LoRA	✗	8.82	9.62	8.77	90.38	8.61	7.34	**8.72**	92.84
	Ours	✓	8.91	6.07	9.13	94.02	9.14	5.47	9.42	94.87
	+ LoRA	✓	**8.30**	**5.22**	**8.53**	**94.87**	**8.46**	**4.89**	8.73	**95.31**

contextual ASR baseline. Incorporating keyword prompts further reduces WERs to 9.0%/11.2%, improving by 3.6%/4.1%. Notably, B-WER drops from 10.8% to 5.8% on L95 and from 14.9% to 8.3% on S95, while Recall improves from 89.4% to 94.4% and from 85.3% to 92.0%. U-WER remains stable, indicating effective use of keyword prompts without harming general transcription. Fine-tuning the LLM with LoRA adapters further improves results. On L95, WER decreases from 9.4% to 8.7%, and to 8.4% when keyword prompts are used. Compared to prior contextual ASR systems, including CPP [42], LCB-Net [43], and the SlideSpeech baseline, our LLM-based approach achieves significantly lower WER, highlighting its superior capability in leveraging context for accurate transcription.

3) Contextual Biasing Speech Recognition:

a) Datasets: We use the LibriSpeech corpus for training and evaluation, following prior work [44], [45]. The LLM-based ASR model is trained on the full 960-hour training set using the official WavLM Large as the pre-trained encoder. Evaluation is conducted on the standard dev-clean/dev-other and test-clean/test-other sets. For contextual ASR, we adopt the artificial biasing list from [44], where the 5,000 most frequent training words are labeled as common, and the rest as rare. Each test-time biasing list includes rare words from the reference and distractors sampled from the 209.2K rare-word vocabulary. Lists of size $N = 100, 500, 1000, 2000$ are constructed by varying the number of distractors.

b) Model setup: We fully fine-tune the official WavLM Large model (315.5M parameters) on the 960-hour LibriSpeech training set using CTC loss. The fine-tuned model serves as the speech encoder. The rest of the architecture mirrors the visual contextual ASR setup, employing Vicuna-7B (6.7B) as the LLM decoder and a lightweight linear projector (15.7M). The training recipes and model checkpoints can be found here[5].

c) Experiments with different biasing lists: Table VII presents the performance of our CTC-assisted LLM-based contextual ASR (CASR) model on LibriSpeech, evaluated using WER, B-WER, and U-WER. For non-contextual ASR, using the pre-trained WavLM Large encoder yields WERs of 2.13%/4.73% on test-clean/test-other. Fine-tuning slightly improves performance to 2.11%/4.20%, serving as our baseline. In contextual ASR, prompting with an empty string achieves WER/B-WER of 1.96%/9.33% (test-clean) and 4.18%/20.02% (test-other), showing robustness even without explicit hotwords. Supplying ground-truth hotwords further reduces WER/B-WER to 1.13%/2.78% and 2.68%/6.00%, representing the upper-bound performance. In practical settings, where biasing lists mix relevant terms with distractors, best perfor-

[5]https://github.com/X-LANCE/SLAM-LLM/tree/main/examples/contextual_asr

mance is achieved with 100-word lists, reducing WER/B-WER to 1.27%/3.67% and 2.72%/8.02%, corresponding to relative WER/B-WER reductions of 39.81%/63.37% and 35.24%/61.37% over the baseline, while U-WER remains stable. As list size increases, performance degrades slightly, but even with 2,000-word lists, the model achieves 1.38%/4.41% (test-clean) and 3.20%/10.02% (test-other), maintaining notable improvements. These results demonstrate the model's strong capacity to filter and utilize biasing information effectively.

TABLE VII: Performance of LLM-Based Contextual ASR on LibriSpeech test sets. "No bias" indicates adding an empty string, "Bias List" refers to incorporating filtered hotwords from the complete biasing list, and "GT Hotwords" denotes including the exact ground truth hotwords during inference.

WavLM Encoder	Prompt	test-clean			test-other		
		WER	B-WER	U-WER	WER	B-WER	U-WER
Pre-trained	✗	2.13	10.15	1.20	4.73	22.43	2.84
CTC Fine-tuned	✗	2.11	10.02	1.20	4.20	20.76	2.43
CTC Fine-tuned	No bias	1.96	9.33	1.11	4.18	20.02	2.49
	100 Biasing words	1.27	3.67	1.00	2.72	8.02	2.16
	500 Biasing words	1.33	3.92	1.03	3.04	9.04	2.40
	1000 Biasing words	1.33	4.16	1.00	2.99	9.33	2.31
	2000 Biasing words	1.38	4.41	1.03	3.20	10.02	2.47
	GT Hotwords	**1.13**	**2.78**	**0.94**	**2.68**	**6.00**	**2.32**

TABLE VIII: Performance comparison with previous NN-based contextual ASR models utilizing artificial biasing lists proposed in [44] on the Librispeech test sets, with biasing list size N set to 1000.

Model	test-clean			test-other		
	WER	B-WER	U-WER	WER	B-WER	U-WER
CPPNet [45]	3.81	11.40	2.90	8.75	25.30	6.90
Deep Biasing+BPB [46]	3.47	7.70	3.00	7.34	15.80	6.40
$\text{TCPGen}_{\text{+GNN enc.}}$ [47]	3.10	6.70	-	7.90	17.80	-
GA-CTC [48]	2.40	6.30	2.00	6.20	15.20	5.20
$\text{TCPGen}_{\text{+p+phn-awareQ}}$ [49]	2.20	4.60	-	6.00	12.30	-
DB-NNLM [44]	2.14	6.70	1.60	6.35	17.20	5.10
Ours [50]	**1.33**	**4.16**	**1.00**	**2.99**	**9.33**	**2.31**

d) Comparison with Non-LLM-based CASR models: Table VIII shows a performance comparison with various NN-based contextual ASR models using the artificial biasing lists proposed in [44] on the Librispeech test sets, with the biasing list size set to 1000. Our LLM-based method significantly outperforms traditional NN-based approaches.

C. Visual Speech Recognition (VSR)

Visual Speech Recognition (VSR) aims to transcribe speech by analyzing visual cues from a speaker's face, as shown in Figure 4. We explore LLM-based VSR in our SLAM-LLM framework.

a) Datasets: We conduct experiments on the large-scale AVSR benchmark dataset LRS3 [51], which comprises over 400 hours of TED and TEDx videos with aligned subtitles and word boundaries. The dataset is split into three subsets: 119k utterances (407 hours) for pre-training, 32k utterances (30 hours) for train-val, and 1,452 utterances (1 hour) for testing. LRS3 presents a significant challenge due to its wide variation in head poses, lighting, speaking styles, and speakers.

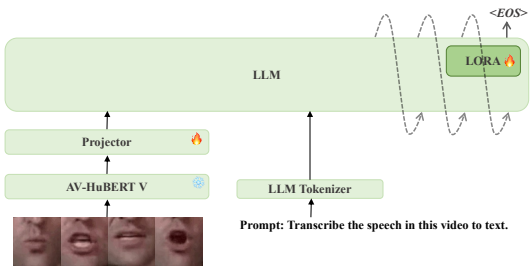

Fig. 4: The model architecture of LLM-based visual speech recognition (VSR) in SLAM-LLM framework.

TABLE IX: Performance of LLM-based VSR model on the LRS3 dataset with SLAM-LLM framework.

Model	Encoder	Decoder	Criterion	Trainable Param.	WER
AV-HuBERT [52]	$\text{AV-HuBERT}_{\text{v}}$	Transformer	CTC+Attention	325	28.6
Ours		$\text{Vicuna}_{\text{v1.5-7B}}$	Decoder-Only	49	28.3

b) Model setup: We use the official AV-HuBERT Large model (477.3M), pre-trained on LRS3 and VoxCeleb2 [53] (1,759 hours of unlabeled data), and fine-tuned on 433 hours of labeled LRS3 data for the VSR task. Vicuna-7B serves as the LLM decoder, with a lightweight linear projector as the adaptor. To enhance performance, we apply LoRA adapters to the key, query, value, and output projection layers in each self-attention module of the LLM, using rank 32, alpha 32, and dropout 0.05. This introduces 33.6M additional trainable parameters. The training recipes and model checkpoints can be found here[6].

c) Experiments: Table IX shows the performance of our LLM-based VSR model trained and evaluated on the LRS3 dataset. When utilizing the same amount of labeled and unlabeled training data, our SLAM-VSR model achieves a better WER of 28.3 compared with the AV-HuBERT baseline model fine-tuned for VSR task, with much less trainable parameters of only 49 million, demonstrating the efficiency of LLM-based structure.

D. Speech-to-Text Translation (S2TT)

a) Model setup: The LLM-based speech-to-text translation (S2TT) model in SLAM-LLM uses a frozen Whisper encoder and Qwen2-7B LLM, with the Q-Former projection as the only trainable component. The encoder extracts speech features, which are compressed and aligned to the LLM input space via the Q-Former. The LLM then generates text outputs based on concatenated auditory and textual embeddings. The training recipes and model checkpoints can be found here[7].

b) Datasets: We conduct experiments using CoVoST-2 [54] for training and MuST-C [55] for zero-shot evaluation.

[6] https://github.com/X-LANCE/SLAM-LLM/tree/main/examples/vsr_LRS3

[7] https://github.com/X-LANCE/SLAM-LLM/tree/main/examples/st_covost2

TABLE X: The prompt design is intended for instruction fine-tuning three tasks: ASR, MMT, and SRT. We designed minimalist yet effective prompts to distinguish tasks.

Task	Instruction	Prediction
ASR	<|en|>	Will it rain tomorrow?
MMT	Will it rain tomorrow?<|de|>	Will it rain tomorrow? <|de|>Regnet es morgen?
SRT	<|de|>	Will it rain tomorrow? <|de|>Regnet es morgen?

TABLE XI: Speech translation BLEU scores on CoVoST-2 and MuST-C datasets. We conducted experiments in German (De), Japanese (Ja), and Chinese (Zh). We use underline to highlight previous SOTA baseline, and use **bold** to highlight surpassing the SOTA performance.

En→X	CoVoST-2				MuST-C$_{(zero\text{-}shot)}$			
	De	Ja	Zh	Avg.	De	Ja	Zh	Avg.
Cascaded ST Methods								
Whisper+NLLB$_{3.3b}$ [56]	-	19.0	32.0	-	-	-	13.5	-
Whisper+Qwen$_{7B\text{-}chat}$ [57]	25.3	22.7	43.6	30.5	22.8	-	-	-
End-to-End Methods								
SALMONN [58]	18.6	22.7	33.1	24.8	-	-	-	-
BLSP-KD [57]	24.4	21.3	41.3	29.0	20.7	-	-	-
SeamlessM4T-V2 [59]	37.0	23.5	34.6	31.7	-	-	18.1	-
Qwen2-Audio [60]	29.9	28.6	45.2	34.5	22.4	7.8	18.3	16.1
Ours-7B	**28.7**	**30.8**	**47.7**	**35.7**	**18.3**	**11.3**	**21.2**	**16.9**

c) Experiments: We adopt a multimodal Chain-of-Thought (CoT) approach to decompose the speech translation (ST) task into two sequential stages: ASR followed by multimodal machine translation (MMT), collectively referred to as SRT (Speech Recognition and Translation), as illustrated in Table X. This formulation enables end-to-end generation of both transcriptions and translations. We apply three-stage SFT with curriculum learning on CoVoST-2, followed by zero-shot evaluation on MuST-C. As shown in Table XI, our model achieves state-of-the-art (SOTA) performance on CoVoST-2 and outperforms existing methods in zero-shot English-to-Chinese translation on MuST-C.

E. Speech Emotion Captioning (SEC)

a) Task setup: Speech Emotion Captioning (SEC) aims to describe speech emotions using natural language, offering a more nuanced alternative to traditional speech emotion recognition (SER), which typically relies on fixed emotion categories and struggles to capture the complexity of human affect. SECap [61] first introduced SEC using an Encoder–Projector–LLM framework to generate rich emotional descriptions. SEC has enabled applications such as PerceptiveAgent [62], which enhances emotional awareness in conversations, and AVI-Talking [63], which improves the expressiveness of talking-face animations. Beyond these, SEC holds promise for a wide range of yet-unexplored applications.

b) Datasets: We trained our model using approximately 40 hours of in-house emotional speech data. Each audio segment in the dataset is annotated with 1 to 3 emotion-related captions, which provide natural language descriptions of the expressed affective content. These captions were carefully curated to capture a diverse range of emotional nuances beyond traditional categorical labels, enabling the model to learn fine-grained emotional representations.

c) Model Setup: SECap [61] achieves state-of-the-art SEC performance using HuBERT [4] as the audio encoder, a Chinese LLaMA-2 decoder, and a Q-Former Bridge-Net to extract emotion-relevant acoustic features. However, it relies on complex training strategies—Speech–Transcription Mutual Information Learning (STMIL) and Speech–Caption Contrastive Learning (SCCL)—to attain its reported gains. To enhance intrinsic emotion perception while simplifying the training pipeline, we replace the audio encoder with emotion2vec [64], a self-supervised model tailored for emotion representation. We adopt Vicuna-7B as the LLM and retain the Q-Former as the projection module. The training recipes and model checkpoints can be found here[8].

d) Experiments: Following the evaluation protocol in SECap, we test the model on the 600 sentences from the EMOSpeech testset released by [61], and we report sentence similarity (SIM) based on MACBERT [65] as the primary metric. Table XII contrasts our LLM-based SEC model in SLAM-LLM with the SECap family and the HTSAT–BART baseline reported in the SECap paper. Without any auxiliary objectives, our model attains a SIM of 71.10%, outperforming the vanilla SECap and narrowing the gap to the most heavily engineered SECap variant. Relative to the HTSAT–BART baseline, the gain exceeds 7 percentage points, underscoring the effectiveness of coupling the SSL emotion2vec encoder with a strong LLM decoder.

TABLE XII: Performance of speech emotion captioning with SLAM-LLM framework. The specific information of the different modules is given in the table. Results other than ours come from the SECap paper.

Model	Encoder	Decoder	SIM(%)
Baseline [61]	HTSAT	BART	63.95
SECap [61]	HuBERT	Chinese-LLaMA2-7B	67.29
+STMIL			68.75
+SCCL			71.95
Ours	emotion2vec	Vicuna-v1.5-7B	71.10

F. Takeaways

From the above experiments, several key takeaways can help guide future research and development of LLM-based speech tasks:

- **Larger models improve performance.** Obviously, both larger speech encoders and larger LLMs consistently lead to better ASR performance.
- **Chat LLMs outperform pre-trained LLMs.** Chat models, such as Vicuna-7B, consistently outperform pre-trained LLMs in ASR tasks, particularly when paired with larger speech encoders.
- **Self-supervised encoders are superior at scale.** Self-supervised encoders outperform supervised encoders

[8] https://github.com/X-LANCE/SLAM-LLM/tree/main/examples/sec_emotioncaps

once they reach a sufficient size. Fine-tuning these self-supervised encoders yields significant performance gains, and though our experiments were limited, we believe fine-tuning LLMs on in-domain text data would also enhance performance.

- **Domain specialization LLMs are good helpers.** For instance, utilizing LLMs trained on low-resource languages can enhance the effectiveness of LLM-based low-resource languages ASR.
- **Whisper models face limitations with truncation.** Whisper models, which pad speech inputs to 30 seconds, experience performance degradation when truncated. Utterance with 30s also significantly increases computational demands during LLM post-training. We recommend using self-supervised models with variable-length embeddings as encoders to mitigate this issue.

More detailed and technique-specific information can be found in sub-tasks research [50], [66], [67] of the SLAM-LLM series.

V. LLM-based Audio and Music Processing

A. Automated audio captioning (AAC)

Automated audio captioning (AAC) aims to generate fine-grained natural language descriptions from input audio, serving as a key task in audio processing. As shown in Figure 5, we focus on two mainstream AAC paradigms with the SLAM-LLM framework: (1) **The vanilla setting**, also known as *fully supervised* audio captioning, where the model is trained on paired audio-text data. (2) **The zero-shot setting**, where the model is trained exclusively on text data and is expected to generate captions for audio clips in a zero-shot manner during inference.

a) Datasets: We use four key audio-text datasets for the audio captioning experiments: Clotho [68], AudioCaps [69], WavCaps [70], and MACS [71]. For Clotho, version 2.1 was used, consisting of audio clips ranging from 15 to 30 seconds in duration. The dataset includes 3,839 training examples, 1,045 validation examples, and 1,045 evaluation examples, with each audio clip accompanied by five captions. AudioCaps contains over 50,000 ten-second audio clips derived from AudioSet [72]. It is split into a training set (49,274 clips, each with one caption), a validation set (494 clips, each with five captions), and a test set (957 clips, each with five captions). WavCaps comprises 403,050 audio clips collected from multiple sources, including AudioSet, BBC Sound Effects, FreeSound, and SoundBible. MACS consists of 3,930 ten-second audio files, each associated with 2 to 5 captions, mainly recorded in three acoustic environments (airport, public square, and park). For our experiments, we used the training sets from Clotho, AudioCaps, and MACS, along with the entire WavCaps dataset for pre-training. In addition, the Clotho training set was augmented using a paraphrasing method, which draws from the back-translation [73] technique used in machine translation to expand the dataset during pre-training.

b) Evaluation Metrics: To evaluate the quality of generated audio captions, we used several standard AAC metrics: METEOR [74], CIDEr [75], SPICE [76], SPIDEr [77], SPIDEr-FL [78] and FENSE [78]. METEOR considers unigram precision, recall, synonyms, and stemming. CIDEr measures n-gram consensus using TF-IDF. SPICE compares semantic graphs of generated and reference captions. SPIDEr linearly combines CIDEr and SPICE for balanced evaluation. SPIDEr-FL further incorporates fluency detection from FENSE, which uses Sentence-BERT for semantic similarity combined with fluency error detection.

c) Model Setup: For vanilla setting, we utilize the frozen EAT model [79] as the audio encoder to extract fine-grained audio representations, which are then downsampled and aligned with LLM embeddings through a linear projector. Specifically, the projector downsamples the 50Hz features to 10Hz using two linear layers, with an intermediate hidden layer dimension of 2048. For decoding, the LLM, Vicuna [10], generates captions based on these concatenated representations and is efficiently fine-tuned using LoRA. During inference, multiple candidate captions are generated through beam search within different beam widths, with the most audio-aligned caption selected as the final output using the CLAP-Refine [80] strategy. The training recipes and model checkpoints can be found here[9].

For zero-shot audio captioning, we utilize the frozen CLAP [81] model as the audio & text encoder. During training, raw captions are first encoded by the text branch of the CLAP model, and then get aligned with the LLM through a two-layer linear projector. The LLM, efficiently fine-tuned using the LoRA [82] method, learns to re-construct the ground-truth caption conditioned on the mapped CLAP latent and an encoded prompt retrieved from a datastore. During inference, the text branch is replaced by the audio branch of the CLAP model, and the system describes audio clips in a zero-shot manner. Projection-based decoding [83] and retrieval-augmented generation (RAG) are employed to reduce the modality gap and improve caption quality. Audio embeddings are projected onto the text embedding space, while similar captions retrieved from a datastore are used as prompts to guide the LLM. The training recipes and model checkpoints can be found here [10].

d) Vanilla Audio Captioning Experiments: For the vanilla audio captioning system, we trained our model on the pre-training dataset, followed by fine-tuning on the AudioCaps and Clotho datasets individually. Furthermore, we conducted a comprehensive ablation study on these two datasets to evaluate the impact of each component.

Table XIII compares the performance of our model with previous SOTA AAC systems. We include three models using non-LLM-based decoding: EnCLAP [84], WavCaps [70], and Wu et al. [85]—and two employing LLM-based decoding—Tang et al. [86] and LOAE [87]. Overall, LLM-based models tend to produce higher-quality captions, likely due to their stronger reasoning capabilities. Compared to previous models, our proposed system demonstrates consistent improvements across both the Clotho and AudioCaps datasets. On Clotho, it achieves the highest scores across all AAC metrics, with a notable improvement in FENSE (54.0%), surpassing both the LOAE

[9] https://github.com/X-LANCE/SLAM-LLM/tree/main/examples/slam_aac

[10] https://github.com/X-LANCE/SLAM-LLM/tree/main/examples/drcap_zeroshot_aac

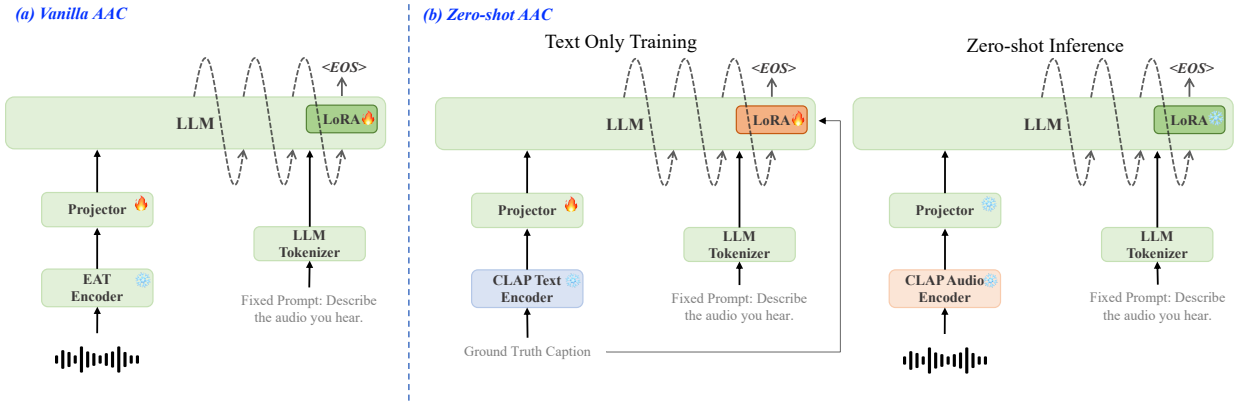

Fig. 5: (a). LLM-based vanilla automated audio captioning in SLAM-LLM framework (b). LLM-based zero-shot automated audio captioning in SLAM-LLM framework

TABLE XIII: Performance comparison of Vanilla *(fully supervised)* AAC models on Clotho and AudioCaps evaluation split. The pre-training datasets used include AudioCaps (AC), Clotho (CL), WavCaps (WC), MACS (MA), LibriSpeech (LS) [30], and GigaSpeech (GS) [33]. Metrics reported are METEOR (MT), CIDEr (CD), SPICE (SC), SPIDEr (SD), SPIDEr-FL (SF), and FENSE (FS). CL_C and CL_P denote the Clotho training set augmented with the ChatGPT Mix-up method and the paraphrasing approach, respectively. All metrics are reported with higher values indicating better performance.

Model	Pre-training Data	Clotho Evaluation (%)						AudioCaps Evaluation (%)					
		MT	CD	SC	SD	SF	FS	MT	CD	SC	SD	SF	FS
EnCLAP-large [84]	AC+CL	18.6	46.4	13.3	29.9	28.9*	50.7*	25.5	80.3	18.8	49.5	49.9*	65.5*
WavCaps† [70]	AC+CL+WC	18.5	48.8	13.3	31.0	29.6*	50.1*	25.0	78.7	18.2	48.5	48.3*	64.2*
Wu et al. [85]	AC+CL_C	19.3	50.6	14.6	32.6	32.6	53.6	-	-	-	-	-	-
Tang et al. [86]	AC+CL+WC+LS+GS	-	-	-	31.8	-	-	-	-	-	50.6	-	-
LOAE [87]	AC+CL+WC	19.7	51.4	14.7	33.1	32.8	53.2	26.7	81.6	19.3	50.5	50.4	66.2
Ours	AC+CL_P+WC+MA	**19.7**	**51.5**	**14.8**	**33.2**	**33.0**	**54.0**	**26.8**	**84.1**	**19.4**	**51.8**	**51.5**	**66.8**

*For open-source models, we evaluated metrics not reported in the original papers using our evaluation split.
†The best performance of WavCaps on both datasets is selected for comparison (model architectures may differ).

and Wu et al. models. On AudioCaps, the improvements are even more pronounced: our model attains a CIDEr score of 84.1%, significantly exceeding LOAE (81.6%), and achieves a FENSE score of 66.8%, outperforming all other models.

Table XIV shows the results of a comprehensive ablation study conducted on the AudioCaps and Clotho. The study explored the impact of different audio encoders, versions of audio encoders (pre-trained vs. fine-tuned on AudioSet), whether pre-training was performed, the use of parameter-efficient fine-tuning (e.g., LoRA), and different decoding strategies (beam search vs. CLAP-Refine). Results show that fine-tuned audio encoders, particularly those with strong performance in classification tasks, significantly enhance AAC quality. Pre-training the LLM-based AAC model further boosts performance, while LoRA improves training efficiency. Additionally, the CLAP-Refine decoding method consistently improves caption quality by enhancing alignment with input audio, leading to higher CIDEr and SPIDEr scores.

e) Zero-shot Audio Captioning Experiments: For zero-shot audio captioning, we conducted experiments in both in-domain and cross-domain scenarios to evaluate the performance. For in-domain scenario, systems are trained and evaluated on the same dataset following the standard train & val & test split. For cross-domain scenario, systems are trained on the training set of the source dataset and evaluated on the evaluation set of the target dataset. Specifically, for Clotho Evaluation, the source dataset is AudioCaps and for AudioCaps Evaluation, the source dataset is Clotho.

TABLE XIV: Ablation study on AudioCaps and Clotho. *PT* indicates a pre-trained model, *FT* denotes a fine-tuned version. BS denotes beam search decoding, CR represents CLAP-Refine.

Dataset	Audio Encoder	Pre-Trained Model	PEFT (LoRA)	Decoding	Metrics (%)			
					MT	CD	SC	SD
AudioCaps	BEATs_{PT}	✗	✗	BS	23.2	69.6	17.2	43.4
	BEATs_{FT}	✗	✗	BS	23.9	69.8	17.3	43.5
	EAT_{PT}	✗	✗	BS	23.5	67.0	17.3	42.2
	EAT_{FT}	✗	✗	BS	25.0	75.3	18.5	46.9
	EAT_{FT}	✗	✓	BS	26.0	80.0	18.3	49.2
	EAT_{FT}	✓	✓	BS	26.1	82.7	**19.5**	51.1
	EAT_{FT}	✗	✓	CR	26.6	81.4	19.4	50.4
	EAT_{FT}	✓	✓	CR	**26.8**	**84.1**	19.4	**51.8**
Clotho	EAT_{FT}	✗	✗	BS	17.1	34.1	11.3	22.7
	EAT_{FT}	✗	✓	BS	17.0	38.9	12.0	25.4
	EAT_{FT}	✓	✓	BS	19.2	49.8	14.4	32.1
	EAT_{FT}	✗	✓	BS	19.4	51.0	14.6	32.8
	EAT_{FT}	✓	✓	CR	**19.7**	**51.5**	**14.8**	**33.2**

Table XV presents a performance comparison between our zero-shot AAC model and previous state-of-the-art. ZerAuCap [88] uses CLAP to guide the LLM to generate descriptions,

TABLE XV: Performance comparison of zero-shot audio captioning systems. For metrics, MT: METEOR, CD: CIDEr, SP: SPICE, SD: SPIDEr, FS: FENSE. Higher values indicate better performance for all metrics.

Method	Scenario	Clotho Evaluation (%)					AudioCaps Evaluation (%)				
		MT	CD	SP	SD	FS	MT	CD	SP	SD	FS
ZerAuCap [88]	In Domain	9.4	14.0	5.3	9.7	-	12.3	28.1	8.6	18.3	-
WSAC [89]		17.4	37.1	12.3	24.7	-	24.1	63.3	17.3	40.3	-
Zhang *et al.* [90]		17.5	41.1	12.2	26.7	48.8	22.0	64.4	15.6	40.0	-
Ours		**18.2**	**43.8**	**13.3**	**28.5**	**53.0**	**25.3**	**70.5**	**18.0**	**44.2**	**66.2**
WSAC [89]	Cross Domain	12.0	20.6	8.2	14.4	-	17.3	25.6	12.0	18.8	-
Zhang *et al.* [90]		13.2	24.8	9.3	17.1	-	18.2	33.7	12.4	23.0	52.1
Ours		**15.0**	**33.3**	**10.4**	**21.8**	**52.2**	**22.9**	**44.3**	**17.0**	**30.6**	**62.6**

TABLE XVI: Ablation study on AudioCaps (in-domain) and AudioCaps ⇒ Clotho (cross-domain) for our zero-shot AAC model DRCap. PD donates Projection Decoding.

Main Components	AudioCaps (%)					AudioCaps⇒Clotho (%)				
	MT	CD	SP	SD	FS	MT	CD	SP	SD	FS
Ours	**25.5**	**70.5**	18.0	**44.2**	**66.2**	**15.0**	**33.3**	**10.4**	**21.8**	**52.2**
- w/o RAG	25.0	69.2	**18.4**	43.7	65.5	14.2	30.1	10.2	20.1	51.3
- w/o LoRA	23.7	64.7	16.4	40.5	64.1	14.1	29.8	9.8	19.8	51.1
- w/o PD	19.9	31.2	13.4	22.3	55.4	13.2	22.8	8.6	15.7	46.4

WSAC [89] trains a text decoder using the prefix language modeling paradigm conditioned on CLAP embeddings, while Zhang *et al.* [90] crafts soft and hard prompts to bridge the modality gap between audio and text embeddings of CLAP. As shown in Table XV, our model surpasses all competitive zero-shot audio captioning systems in in-domain scenarios by a large margin and is comparable with other fully supervised methods. For cross-domain scenarios, it achieves state-of-the-art results across all metrics, highlighting its robust domain-transfer capability. Furthermore, we found that our model outperforms other methods in terms of the FENSE [78] score in both two scenarios. We hypothesize that this advantage is due to its ability to utilize the semantically rich joint multi-modal space of CLAP, which allows it to generate more refined captions.

Table XVI shows a comprehensive ablation study on different core components of our zero-shot AAC model on both in-domain and cross-domain scenarios. The study explored the contribution of the retrieval-augmented generation (RAG), the use of LoRA adapter, and the effectiveness of the projection decoding (PD). Results show that projection-based decoding helps mitigate the modality gap, the LoRA enhances training efficiency and quality, while the retrieval-augmented generation can also boost model's performance, especially in the cross-domain scenario.

B. Music Captioning (MC)

a) Datasets: We utilize the LP-MusicCaps-MC datasets [91] for training, and conduct testing on the standard test set of LP-MusicCaps-MC. For all training, the music audio is sliced into 10-second clips. Since the raw audio of the LP-MusicCaps-MSD dataset is difficult to access, we are only able to train our models on the LP-MusicCaps-MC dataset, which is a small subset of LP-MusicCaps dataset. However, we demonstrate in the experiments that even models trained only on LP-MusicCaps-MC can achieve comparable results to existing models [91], [92] pre-trained on LP-MusicCaps-MSD.

b) Model setup: We use a pre-trained model as the music encoder and Vicuna-7b-v1.5 [10] as the LLM, both of which are kept frozen during training. We use a linear projector, and the projector is the only trainable component. We consider two different types of music encoders. The first is the **frame-wise encoder**, which extracts features with a temporal dimension from music audio, typically trained by self-supervised learning (SSL), such as MERT [6], MusicFM [93], and MuQ [94]. The second is the **sequence-wise encoder**, which extracts features directly from the music to extract a fixed dimensional feature (e.g., 512 or 1024 dimensions), typically trained by contrastive learning, such as Laion-CLAP [81], Microsoft-CLAP [95], and MuQ-MuLan [94]. For frame-wise encoder, the projector downsamples frame-wise features into 0.5hz, which is the final input fed to LLM, for example, 10s of music corresponds to 5 tokens. For sequence-wise encoder, we do not downsample and feed directly into the LLM after a linear layer projection, which means that the sequence-wise encoded features have only 1 token. The training recipes and model checkpoints can be found here[11].

c) Experiments: In Table XVII, we compare the effect of different music encoders on the music captioning task with existing models. Note that the SLAM-LLM models in the table are all trained only on the small LP-MusicCaps-MC data. For the frame-wise encoder, MuQ is better than MusicFM and MERT, which is consistent with the basic performance of these SSL models. For the sequence-wise encoder, surprisingly, even though the sequence-wise features have only 1 token, they generally work better than the frame-wise features. Among them, MuQ-MuLan achieves the best performance in the four metrics BLEU, METEOR, ROUGE-S and BERT-S. This means that models trained using contrastive learning, even with only one embedding, are more suitable for music captioning tasks than encoders trained using SSL. We also note that although SLAM-LLM uses only a small amount of data from LP-MusicCaps-MC, it achieves comparable results with models such as MusCaps [96], LP-MusicCaps [91], and MusiLingo [92], which are trained using additional pre-training data.

C. Takeaways

From the above experiments, several key takeaways can help guide future research and development of LLM-based audio and music tasks:

- **LLM-based models outperform non-LLM baselines.** Our explorations further advance this trend in AAC tasks, achieving SOTA results on both Clotho and AudioCaps.
- **Fine-tuned encoders matter.** Audio encoders like EAT, when fine-tuned on classification tasks, consistently outperform pre-trained or less optimized counterparts.
- **Pre-training and PEFT enhance quality and efficiency.** Model pre-training combined with LoRA yields better captions with lower training cost.

[11] https://github.com/X-LANCE/SLAM-LLM/tree/main/examples/mc_musiccaps

TABLE XVII: The results of music caption experiments, all evaluated on the LP-MusicCaps-MC test set. The rows highlighted in gray represent models pre-trained using additional or private data.

Model		Train Dataset	BLEU	METEOR	ROUGE-S	BERT-S
MusCaps		Private	10.2	17.0	22.2	83.5
LP-MusicCaps		MSD+MTT+MC	14.7	22.4	21.5	87.8
MusiLingo		MSD+MC	30.8	21.6	21.7	86.8
Ours	***Encoder***$_{\text{Frame-wise}}$					
	MERT	MC	22.2	11.6	16.8	86.3
	MusicFM	MC	25.1	18.6	19.2	86.2
	MuQ	MC	27.0	19.2	19.1	86.8
	Encoder$_{\text{Sequence-wise}}$					
	Laion-CLAP	MC	25.9	19.6	20.9	86.6
	Microsoft-CLAP$_{2023}$	MC	27.5	20.3	**21.1**	86.8
	MuQ-MuLan	MC	**27.9**	**20.5**	**21.1**	**86.9**

- **RAG boost model's performance.** By enabling access to external knowledge for describing unseen soundscapes, RAG enhances model's results.
- **Projection decoding narrows the modality gap.** To bridge the modality gap in CLAP, projection-based decoding proves effective, while direct use of the audio encoder in zero-shot settings performs unsatisfactorily.
- **Even a single token can be sufficient.** In music captioning tasks, where information is relatively sparse, a strong encoder (MuQ) can effectively condense the relevant content into a single token and convey the content to the LLM decoder.
- **Optimal projector choice varies based on the task type.** Linear Projectors are preferred for tasks requiring strict monotonic alignment and temporal sequence preservation, such as ASR and VSR, while Q-Former bridges are more effective for tasks demanding global semantic perception and cross-modal context awareness, such as SEC and AAC.

More detailed and technique-specific information can be found in sub-tasks research [80], [97] of the SLAM-LLM series.

VI. Conclusion

SLAM-LLM presents a significant advancement in the field of multimodal large language models by providing a modular, flexible, and open-source framework specifically tailored for speech, language, audio, and music processing. Through its encoder–projector–LLM architecture, SLAM-LLM enables efficient customization and deployment across a wide range of tasks including ASR, SEC, AAC, and many other tasks. Experimental results demonstrate that recipes in SLAM-LLM not only achieve competitive or state-of-the-art performance on several benchmarks but also give insights for the LLM-based audio processing community. By lowering the entry barrier and promoting community collaboration, SLAM-LLM is poised to accelerate research and innovation in audio-language modeling and unlock new possibilities in multimodal AI.

References

[1] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Proc. NeurIPS*, 2023.

[2] A. Awadalla, I. Gao, J. Gardner, J. Hessel, Y. Hanafy, W. Zhu, K. Marathe *et al.*, "OpenFlamingo: An open-source framework for training large autoregressive vision-language models," *CoRR*, vol. abs/2308.01390, 2023.

[3] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proc. ICML*, 2023.

[4] W.-N. Hsu, B. Bolte, Y.-H. H. Tsai, K. Lakhotia, R. Salakhutdinov, and A. Mohamed, "HuBERT: Self-supervised speech representation learning by masked prediction of hidden units," in *Proc. TASLP*, 2021.

[5] S. Chen, Y. Wu, C. Wang, S. Liu, D. Tompkins, Z. Chen, and F. Wei, "BEATs: Audio pre-training with acoustic tokenizers," in *Proc. ICML*, 2023.

[6] Y. Li, R. Yuan, G. Zhang, Y. Ma, X. Chen, H. Yin, C. Xiao *et al.*, "MERT: acoustic music understanding model with large-scale self-supervised training," in *Proc. ICLR*. OpenReview.net, 2024.

[7] J. Li, D. Li, S. Savarese, and S. Hoi, "BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models," in *Proc. ICML*, 2023.

[8] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and efficient foundation language models," *CoRR*, vol. abs/2302.13971, 2023.

[9] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov *et al.*, "LLaMA 2: Open foundation and fine-tuned chat models," *CoRR*, vol. abs/2307.09288, 2023.

[10] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu *et al.*, "Vicuna: An open-source chatbot impressing GPT-4 with 90%* ChatGPT quality," *https://vicuna.lmsys.org*, 2023.

[11] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang *et al.*, "Qwen technical report," *arXiv preprint arXiv:2309.16609*, 2023.

[12] meta llama, "llama-recipes," https://github.com/meta-llama/llama-recipes, 2024.

[13] R. Zhang, J. Han, C. Liu, P. Gao, A. Zhou, X. Hu, S. Yan, P. Lu, H. Li, and Y. Qiao, "Llama-adapter: Efficient fine-tuning of language models with zero-init attention," *arXiv preprint arXiv:2303.16199*, 2023.

[14] P. Gao, J. Han, R. Zhang, Z. Lin, S. Geng, A. Zhou, W. Zhang, P. Lu, C. He, X. Yue *et al.*, "LLaMA-Adapter v2: Parameter-efficient visual instruction model," in *Proc. ICLR*, 2024.

[15] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, "Learning transferable visual models from natural language supervision," in *Proc. ICML*. PmLR, 2021, pp. 8748–8763.

[16] M. Oquab, T. Darcet, T. Moutakanni, H. V. Vo, M. Szafraniec, V. Khalidov, P. Fernandez *et al.*, "DINOv2: Learning robust visual features without supervision," *TMLR*, 2024.

[17] Y. Zheng, R. Zhang, J. Zhang, Y. Ye, Z. Luo, Z. Feng, and Y. Ma, "LlamaFactory: Unified efficient fine-tuning of 100+ language models," in *Proc. ACL*, 2024.

[18] S. Diao, R. Pan, H. Dong, K. Shum, J. Zhang, W. Xiong, and T. Zhang, "LMFlow: An extensible toolkit for finetuning and inference of large foundation models," in *Proc. NAACL*, 2024, pp. 116–127.

[19] D. Zhu, J. Chen, X. Shen, X. Li, and M. Elhoseiny, "MiniGPT-4: Enhancing vision-language understanding with advanced large language models," in *Proc. ICLR*, 2024.

[20] B. Zhou, Y. Hu, X. Weng, J. Jia, J. Luo, X. Liu, J. Wu, and L. Huang, "TinyLLaVA: A framework of small-scale large multimodal models," *CoRR*, vol. abs/2402.14289, 2024.

[21] J.-B. Alayrac, J. Donahue, P. Luc, A. Miech, I. Barr, Y. Hasson, K. Lenc, A. Mensch, K. Millican, M. Reynolds *et al.*, "Flamingo: a visual language model for few-shot learning," in *Proc. NeurIPS*, 2022.

[22] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, S. Paul, and B. Bossan, "Peft: State-of-the-art parameter-efficient fine-tuning methods," https://github.com/huggingface/peft, 2022.

[23] Y. Zhao, A. Gu, R. Varma, L. Luo, C.-C. Huang, M. Xu, L. Wright *et al.*, "PyTorch FSDP: Experiences on scaling fully sharded data parallel," *Proc. VLDB Endow.*, vol. 16, no. 12, pp. 3848–3860, 2023.

[24] J. Rasley, S. Rajbhandari, O. Ruwase, and Y. He, "Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters," in *Proc. SIGKDD*, 2020, pp. 3505–3506.

[25] L. Dou, Q. Liu, G. Zeng, J. Guo, J. Zhou, X. Mao, Z. Jin, W. Lu, and M. Lin, "Sailor: Open language models for south-East Asia," in *Proc. EMNLP*, 2024, pp. 424–435.

[26] Y. Yang, Z. Song, J. Zhuo, M. Cui, J. Li, B. Yang, Y. Du *et al.*, "Gigaspeech 2: An evolving, large-scale and multi-domain asr corpus

for low-resource languages with automated crawling, transcription and refinement," in *Proc. ACL*, Vienna, 2025.
[27] Z. Zhang, L. Zhou, C. Wang, S. Chen, Y. Wu, S. Liu, Z. Chen, Y. Liu, H. Wang, J. Li *et al.*, "Speak foreign languages with your own voice: Cross-lingual neural codec language modeling," *arXiv preprint arXiv:2303.03926*, 2023.
[28] W. Chen, Z. Ma, R. Yan, Y. Liang, X. Li, R. Xu, Z. Niu, Y. Zhu, Y. Yang, Z. Liu *et al.*, "Slam-omni: Timbre-controllable voice interaction system with single-stage training," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 2262–2282.
[29] S. Chen, C. Wang, Z. Chen, Y. Wu, S. Liu, Z. Chen, J. Li, N. Kanda, T. Yoshioka, X. Xiao *et al.*, "WavLM: Large-scale self-supervised pre-training for full stack speech processing," in *Proc. JSTSP*, 2022.
[30] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "Librispeech: An ASR corpus based on public domain audio books," in *Proc. of ICASSP*, 2015.
[31] J. Kahn, M. Rivière, W. Zheng, E. Kharitonov, Q. Xu, P.-E. Mazaré, J. Karadayi, V. Liptchinsky, R. Collobert, C. Fuegen *et al.*, "Libri-light: A benchmark for asr with limited or no supervision," in *Proc. ICASSP*, 2020.
[32] C. Wang, M. Riviere, A. Lee, A. Wu, C. Talnikar, D. Haziza, M. Williamson, J. Pino, and E. Dupoux, "Voxpopuli: A large-scale multilingual speech corpus for representation learning, semi-supervised learning and interpretation," in *Proc. ACL*, 2021.
[33] G. Chen, S. Chai, G. Wang, J. Du, W.-Q. Zhang, C. Weng, D. Su, D. Povey, J. Trmal, J. Zhang *et al.*, "Gigaspeech: An evolving, multi-domain ASR corpus with 10,000 hours of transcribed audio," in *Proc. Interspeech*, 2021.
[34] P. Zhang, G. Zeng, T. Wang, and W. Lu, "TinyLLaMA: An open-source small language model," *arXiv preprint arXiv:2401.02385*, 2024.
[35] W. Han, Z. Zhang, Y. Zhang, J. Yu, C.-C. Chiu, J. Qin, A. Gulati, R. Pang, and Y. Wu, "Contextnet: Improving convolutional neural networks for automatic speech recognition with global context," in *Proc. Interspeech*, 2020, pp. 3610–3614.
[36] A. Gulati, J. Qin, C.-C. Chiu, N. Parmar, Y. Zhang, J. Yu, W. Han, S. Wang, Z. Zhang, Y. Wu *et al.*, "Conformer: Convolution-augmented Transformer for speech recognition," in *Proc. Interspeech*, 2020.
[37] Y. Peng, S. Dalmia, I. Lane, and S. Watanabe, "Branchformer: Parallel mlp-attention architectures to capture local and global context for speech recognition and understanding," in *Proc. ICML*, 2022.
[38] Z. Yao, L. Guo, X. Yang, W. Kang, F. Kuang, Y. Yang, Z. Jin, L. Lin, and D. Povey, "Zipformer: A faster and better encoder for automatic speech recognition," in *Proc. ICLR*, 2024.
[39] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," in *Proc. NeurIPS*, 2020.
[40] Y. Peng, J. Tian, W. Chen, S. Arora, B. Yan, Y. Sudo, M. Shakeel, K. Choi, J. Shi, X. Chang, J. weon Jung, and S. Watanabe, "OWSM v3.1: Better and faster open whisper-style speech models based on e-branchformer," in *Proc. Interspeech*, 2024, pp. 352–356.
[41] X. Shi, Q. Feng, and L. Xie, "The ASRU 2019 Mandarin-English code-switching speech recognition challenge: Open datasets, tracks, methods and results," *arXiv preprint arXiv:2007.05916*, 2020.
[42] H. Wang, F. Yu, X. Shi, Y. Wang, and S. Zhang, "SlideSpeech: A large-scale slide-enriched audio-visual corpus," in *Proc. ICASPP*, 2024.
[43] F. Yu, H. Wang, X. Shi, and S. Zhang, "LCB-NET: Long-context biasing for audio-visual speech recognition," in *Proc. ICASPP*, 2024.
[44] D. Le, M. Jain, G. Keren, S. Kim, Y. Shi, J. Mahadeokar, J. Chan *et al.*, "Contextualized streaming end-to-end speech recognition with trie-based deep biasing and shallow fusion," in *Proc. Interspeech*, 2021, pp. 1772–1776.
[45] K. Huang, A. Zhang, Z. Yang, P. Guo, B. Mu, T. Xu, and L. Xie, "Contextualized end-to-end speech recognition with contextual phrase prediction network," in *Proc. Interspeech*, 2023, pp. 4933–4937.
[46] Y. Sudo, M. Shakeel, Y. Fukumoto, Y. Peng, and S. Watanabe, "Contextualized automatic speech recognition with attention-based bias phrase boosted beam search," in *Proc. ICASSP*, 2024, pp. 10 896–10 900.
[47] G. Sun, C. Zhang, and P. Woodland, "Tree-constrained pointer generator with graph neural network encodings for contextual speech recognition," in *Proc. Interspeech*, 2022, pp. 2043–2047.
[48] J. Tang, K. Kim, S. Shon, F. Wu, and P. Sridhar, "Improving ASR contextual biasing with guided attention," in *Proc. ICASSP*, 2024.
[49] H. Futami, E. Tsunoo, Y. Kashiwagi, H. Ogawa, S. Arora, and S. Watanabe, "Phoneme-aware encoding for prefix-tree-based contextual ASR," in *Proc. ICASSP*, 2024.
[50] G. Yang, Z. Ma, Z. Gao, S. Zhang, and X. Chen, "CTC-Assisted LLM-Based Contextual ASR," *Proc. SLT*, 2024.
[51] T. Afouras, J. S. Chung, and A. Zisserman, "LRS3-TED: a large-scale dataset for visual speech recognition," *arXiv preprint*, 2018.
[52] B. Shi, W.-N. Hsu, K. Lakhotia, and A. Mohamed, "Learning audio-visual speech representation by masked multimodal cluster prediction," in *Proc. ICLR*, 2022.
[53] J. S. Chung, A. Nagrani, and A. Zisserman, "Voxceleb2: Deep speaker recognition," in *Proc. Interspeech*, 2018, pp. 1086–1090.
[54] C. Wang, A. Wu, J. Gu, and J. Pino, "CoVoST 2 and massively multilingual speech translation," in *Proc. Interspeech*, 2021, pp. 2247–2251.
[55] M. A. Di Gangi, R. Cattoni, L. Bentivogli, M. Negri, and M. Turchi, "Must-c: a multilingual speech translation corpus," in *Proc. NAACL*, 2019, pp. 2012–2017.
[56] M. R. Costa-jussà, J. Cross, O. Çelebi, M. Elbayad, K. Heafield, K. Heffernan, E. Kalbassi, J. Lam, D. Licht, J. Maillard *et al.*, "No language left behind: Scaling human-centered machine translation," *arXiv preprint arXiv:2207.04672*, 2022.
[57] C. Wang, M. Liao, Z. Huang, and J. Zhang, "BLSP-KD: Bootstrapping language-speech pre-training via knowledge distillation," *arXiv preprint arXiv:2405.19041*, 2024.
[58] C. Tang, W. Yu, G. Sun, X. Chen, T. Tan, W. Li, L. Lu, Z. Ma, and C. Zhang, "SALMONN: Towards generic hearing abilities for large language models," in *Proc. ICLR*, 2024.
[59] L. Barrault, Y.-A. Chung, M. C. Meglioli, D. Dale, N. Dong, M. Duppenthaler, P.-A. Duquenne, B. Ellis, H. Elsahar, J. Haaheim *et al.*, "Seamless: Multilingual expressive and streaming speech translation," *arXiv preprint arXiv:2312.05187*, 2023.
[60] Y. Chu, J. Xu, Q. Yang, H. Wei, X. Wei, Z. Guo, Y. Leng, Y. Lv, J. He, J. Lin *et al.*, "Qwen2-audio technical report," *arXiv preprint arXiv:2407.10759*, 2024.
[61] Y. Xu, H. Chen, J. Yu, Q. Huang, Z. Wu, S.-X. Zhang, G. Li, Y. Luo, and R. Gu, "SECap: Speech emotion captioning with large language model," *Proc. AAAI*, 2024.
[62] H. Yan, Y. Zhu, K. Zheng, B. Liu, H. Cao, D. Jiang, and L. Xu, "Talk with human-like agents: Empathetic dialogue through perceptible acoustic reception and reaction," in *Proc. ACL*, 2024, pp. 15 009–15 022.
[63] Y. Sun, W. Chu, H. Zhou, K. Wang, and H. Koike, "AVI-Talking: Learning audio-visual instructions for expressive 3d talking face generation," *IEEE Access*, 2024.
[64] Z. Ma, Z. Zheng, J. Ye, J. Li, Z. Gao, S. Zhang, and X. Chen, "emotion2vec: Self-supervised pre-training for speech emotion representation," *Proc. ACL*, 2024.
[65] Y. Cui, W. Che, T. Liu, B. Qin, and Z. Yang, "Pre-training with whole word masking for chinese bert," *Proc. TASLP*, 2021.
[66] Z. Ma, G. Yang, Y. Yang, Z. Gao, J. Wang, Z. Du, F. Yu, Q. Chen, S. Zheng, S. Zhang *et al.*, "An embarrassingly simple approach for llm with strong asr capacity," *arXiv preprint arXiv:2402.08846*, 2024.
[67] G. Yang, Z. Ma, F. Yu, Z. Gao, S. Zhang, and X. Chen, "MaLa-ASR: Multimedia-assisted llm-based asr," *Proc. Interspeech*, 2024.
[68] K. Drossos, S. Lipping, and T. Virtanen, "Clotho: An audio captioning dataset," in *Proc. ICASSP*. IEEE, 2020, pp. 736–740.
[69] C. D. Kim, B. Kim, H. Lee, and G. Kim, "Audiocaps: Generating captions for audios in the wild," in *Proc. NAACL*, 2019, pp. 119–132.
[70] X. Mei, C. Meng, H. Liu, Q. Kong, T. Ko, C. Zhao, M. D. Plumbley, Y. Zou, and W. Wang, "WavCaps: A chatgpt-assisted weakly-labelled audio captioning dataset for audio-language multimodal research," *arXiv preprint arXiv:2303.17395*, 2023.
[71] I. Martín-Morató and A. Mesaros, "What is the ground truth? reliability of multi-annotator data for audio tagging," in *Proc. EUSIPCO*, 2021.
[72] J. F. Gemmeke, D. P. Ellis, D. Freedman, A. Jansen, W. Lawrence, R. C. Moore, M. Plakal, and M. Ritter, "Audio set: An ontology and human-labeled dataset for audio events," in *Proc. ICASSP*. IEEE, 2017, pp. 776–780.
[73] R. Sennrich, B. Haddow, and A. Birch, "Improving neural machine translation models with monolingual data," in *Proc. ACL*, 2016, pp. 86–96.
[74] S. Banerjee and A. Lavie, "METEOR: An automatic metric for mt evaluation with improved correlation with human judgments," in *Proc. ACL*, 2005.
[75] R. Vedantam, C. Lawrence Zitnick, and D. Parikh, "CIDEr: Consensus-based image description evaluation," in *Proc. CVPR*, 2015.
[76] P. Anderson, B. Fernando, M. Johnson *et al.*, "SPICE: Semantic propositional image caption evaluation," in *Proc. ECCV*, 2016.
[77] S. Liu, Z. Zhu, N. Ye, S. Guadarrama, and K. Murphy, "Improved image captioning via policy gradient optimization of spider," in *Proc. ICCV*, 2017.

[78] Z. Zhou, Z. Zhang, X. Xu, Z. Xie, M. Wu, and K. Q. Zhu, "Can audio captions be evaluated with image caption metrics?" in *Proc. ICASSP*, 2022.
[79] W. Chen, Y. Liang, Z. Ma, Z. Zheng, and X. Chen, "EAT: Self-supervised pre-training with efficient audio transformer," in *Proc. IJCAI*, 2024.
[80] W. Chen, Z. Ma, X. Li, X. Xu, Y. Liang, Z. Zheng, K. Yu, and X. Chen, "SLAM-AAC: Enhancing audio captioning with paraphrasing augmentation and clap-refine through llms," in *Proc. ICASSP*, 2025, pp. 1–5.
[81] Y. Wu, K. Chen, T. Zhang, Y. Hui, T. Berg-Kirkpatrick, and S. Dubnov, "Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation," in *Proc. ICASSP*, 2023.
[82] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.
[83] W. Li, L. Zhu, L. Wen, and Y. Yang, "DeCap: Decoding CLIP latents for zero-shot captioning via text-only training," in *Proc. ICLR*, 2023.
[84] J. Kim, J. Jung, J. Lee, and S. H. Woo, "EnCLAP: Combining neural audio codec and audio-text joint embedding for automated audio captioning," in *Proc. ICASSP*, 2024.
[85] S.-L. Wu, X. Chang, G. Wichern, J.-w. Jung, F. Germain, J. Le Roux, and S. Watanabe, "BEATs-based audio captioning model with INSTRUCTOR embedding supervision and ChatGPT mix-up," *Tech. Rep., DCASE Challenge*, 2023.
[86] C. Tang, W. Yu, G. Sun, X. Chen, T. Tan, W. Li, L. Lu, Z. Ma, and C. Zhang, "Extending large language models for speech and audio captioning," in *Proc. ICASSP*, 2024.
[87] J. Liu, G. Li, J. Zhang, H. Dinkel, Y. Wang, Z. Yan, Y. Wang, and B. Wang, "Enhancing automated audio captioning via large language models with optimized audio encoding," in *Proc. Interspeech*, 2024, pp. 1135–1139.
[88] S. Deshmukh, R. Singh, and B. Raj, "Domain adaptation for contrastive audio-language models," in *Proc. Interspeech*, 2024, pp. 1680–1684.
[89] T. Kouzelis and V. Katsouros, "Weakly-supervised automated audio captioning via text only training," *arXiv preprint arXiv:2309.12242*, 2023.
[90] Y. Zhang, X. Xu, R. Du, H. Liu, Y. Dong, Z.-H. Tan, W. Wang, and Z. Ma, "Zero-shot audio captioning using soft and hard prompts," *IEEE TASLP*, vol. 33, pp. 2045–2058, 2025.
[91] S. Doh, K. Choi, J. Lee, and J. Nam, "Lp-musiccaps: Llm-based pseudo music captioning," *arXiv preprint arXiv:2307.16372*, 2023.
[92] Z. Deng, Y. Ma, Y. Liu, R. Guo, G. Zhang, W. Chen, W. Huang, and E. Benetos, "MusiLingo: Bridging music and text with pre-trained language models for music captioning and query response," in *Findings of NAACL*, 2024, pp. 3643–3655.
[93] M. Won, Y.-N. Hung, and D. Le, "A foundation model for music informatics," in *Proc. ICASSP*. IEEE, 2024, pp. 1226–1230.
[94] H. Zhu, Y. Zhou, H. Chen, J. Yu, Z. Ma, R. Gu, Y. Luo, W. Tan, and X. Chen, "MuQ: Self-supervised music representation learning with mel residual vector quantization," *CoRR*, vol. abs/2501.01108, 2025.
[95] B. Elizalde, S. Deshmukh, M. Al Ismail, and H. Wang, "Clap learning audio concepts from natural language supervision," in *Proc. ICASSP*, 2023.
[96] I. Manco, E. Benetos, E. Quinton, and G. Fazekas, "Muscaps: Generating captions for music audio," in *Proc. IJCNN*. IEEE, 2021, pp. 1–8.
[97] X. Li, W. Chen, Z. Ma, X. Xu, Y. Liang, Z. Zheng, Q. Kong, and X. Chen, "Drcap: Decoding clap latents with retrieval-augmented generation for zero-shot audio captioning," in *Proc. ICASSP*, 2025, pp. 1–5.